

Chapter 6: Second-Order Methods

Algorithms for Optimization, 2nd ed. (2026)

Kochenderfer & Wheeler

Lecture Slides

Outline

- 1 Introduction
- 2 Newton's Method
- 3 Secant Method
- 4 Levenberg-Marquardt Algorithm
- 5 Levenberg-Marquardt for Sum of Squares
- 6 Quasi-Newton Methods
- 7 Summary & Exercises

Motivation: Why Second-Order Information?

First-Order Methods

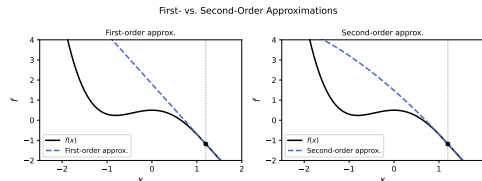
- Use gradient $\nabla f(\mathbf{x})$ to determine direction
- Do *not* directly inform the step size needed to reach a minimum
- Can take many small steps in curved regions

Second-Order Methods

- Use the Hessian $\mathbf{H}(\mathbf{x}) = \nabla^2 f(\mathbf{x})$
- Build a *quadratic* local model of f
- Quadratic models have an analytically optimal step
- Can converge in a single step on quadratic functions

Key Idea

A bowl-shaped quadratic approximation has a unique



Quadratic Approximation (Taylor Expansion)

Univariate: Second-Order Taylor Expansion (Eq. 6.1)

$$q(x) = f(x^{(k)}) + (x - x^{(k)})f'(x^{(k)}) + \frac{(x - x^{(k)})^2}{2} f''(x^{(k)})$$

- $x^{(k)}$ – current iterate (expansion point)
- $f'(x^{(k)})$ – first derivative (slope) at $x^{(k)}$
- $f''(x^{(k)})$ – second derivative (curvature) at $x^{(k)}$

Multivariate: Second-Order Taylor Expansion

$$q(\mathbf{x}) = f(\mathbf{x}^{(k)}) + (\mathbf{x} - \mathbf{x}^{(k)})^\top \mathbf{g}^{(k)} + \frac{1}{2}(\mathbf{x} - \mathbf{x}^{(k)})^\top \mathbf{H}^{(k)}(\mathbf{x} - \mathbf{x}^{(k)})$$

- $\mathbf{g}^{(k)} = \nabla f(\mathbf{x}^{(k)}) \in \mathbb{R}^n$ – gradient vector
- $\mathbf{H}^{(k)} = \nabla^2 f(\mathbf{x}^{(k)}) \in \mathbb{R}^{n \times n}$ – Hessian matrix

6.1 Newton's Method

Univariate Update Rule (Eq. 6.3–6.4)

Setting $q'(x) = 0$ and solving for the next iterate:

$$x^{(k+1)} \leftarrow x^{(k)} - \frac{f'(x^{(k)})}{f''(x^{(k)})}$$

Multivariate Update Rule (Eq. 6.6)

Setting $\nabla q(\mathbf{x}) = 0$ and solving for $\mathbf{x}$:

$$\mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^{(k)} - \left(\mathbf{H}^{(k)}\right)^{-1} \mathbf{g}^{(k)}$$

- $\mathbf{g}^{(k)} = \nabla f(\mathbf{x}^{(k)})$ – gradient at current iterate
- $\mathbf{H}^{(k)} = \nabla^2 f(\mathbf{x}^{(k)})$ – Hessian at current iterate
- In practice: solve $\mathbf{H}^{(k)} \mathbf{d} = -\mathbf{g}^{(k)}$, then $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \mathbf{d}$

Newton's Method: Algorithm & Convergence I

Algorithm 6.1: Newton's Method

- ➊ **Input:** f , $f'/\nabla f$, $f''/\nabla^2 f$, initial $\mathbf{x}^{(1)}$, tolerance ϵ
- ➋ **Repeat** until $\|\mathbf{g}^{(k)}\| \leq \epsilon$:
 - ➊ Compute $\mathbf{g}^{(k)} = \nabla f(\mathbf{x}^{(k)})$
 - ➋ Compute $\mathbf{H}^{(k)} = \nabla^2 f(\mathbf{x}^{(k)})$
 - ➌ Solve $\mathbf{H}^{(k)}\mathbf{d} = -\mathbf{g}^{(k)}$
 - ➍ Update: $\mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^{(k)} + \mathbf{d}$
- ➌ **Return** $\mathbf{x}^{(k+1)}$

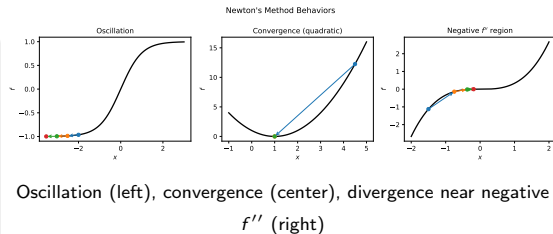
Convergence Properties

- **Quadratic convergence** near a local minimum (error squares at each step)
- Converges in **one step** on strictly quadratic functions
- Often terminated once $x^{(k+1)}$ changes by less than some tolerance ϵ
- **Cost per iteration:** $O(n^3)$ for Hessian inversion (or $O(n^2)$ with structure)

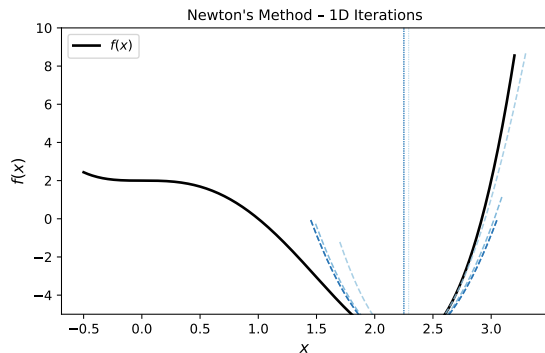
Newton's Method: Algorithm & Convergence III

Potential Problems

- $f''(x^{(k)}) = 0$ or $\mathbf{H}^{(k)}$ singular: undefined step
- $f''(x^{(k)}) < 0$: step goes uphill (oscillation or divergence)
- Not guaranteed to descend: may need line search
- Requires second derivatives (costly to compute)



Newton's Method: Iterations on 1D Function



Quadratic approximations (dashed) converging to minimum.

Key Observations

- Each dashed curve is the quadratic approximation $q(x)$ at the current point
- The next iterate is the minimum of $q(x)$
- Near the minimum, convergence is quadratic
- Far from minimum, behavior can be unpredictable

Termination Criteria

Terminate when *any* of these are satisfied:

- 1 $\|\mathbf{g}^{(k)}\| \leq \epsilon$ (gradient small)
- 2 $\|x^{(k+1)} - x^{(k)}\| \leq \epsilon$ (step small)
- 3 Maximum iterations reached

Newton's Method: Python Implementation

Listing 1: Newton's method (multivariate) in Python

```
1 import numpy as np
2 from numpy.linalg import solve, norm
3
4 def newtons_method(f, grad_f, hess_f, x0, eps=1e-8, max_iter=100):
5     """
6     Newton's method for unconstrained minimization.
7     f          : objective function  $\mathbb{R}^n \rightarrow \mathbb{R}$ 
8     grad_f     : gradient function  $\mathbb{R}^n \rightarrow \mathbb{R}^n$ 
9     hess_f     : Hessian function  $\mathbb{R}^n \rightarrow \mathbb{R}^{\{n \times n\}}$ 
10    x0         : initial point (numpy array)
11    """
12    x = np.array(x0, dtype=float)
13    history = [x.copy()]
14    for k in range(max_iter):
15        g = grad_f(x)
16        if norm(g) < eps:
17            break
18        H = hess_f(x)
19        # Solve  $H*d = -g$  (more stable than inverting H)
20        d = solve(H, -g)
21        x = x + d
22        history.append(x.copy())
23    return x, history
24
25 # --- Example: Booth's function  $f(x) = (x_1+2x_2-7)^2 + (2x_1+x_2-5)^2$  ---
```

Example 6.1: Newton's Method on Booth's Function

Booth's Function

$$f(\mathbf{x}) = (x_1 + 2x_2 - 7)^2 + (2x_1 + x_2 - 5)^2$$

Global minimum at $\mathbf{x}^* = [1, 3]^\top$.

Gradient

$$\nabla f(\mathbf{x}) = \begin{bmatrix} 10x_1 + 8x_2 - 34 \\ 8x_1 + 10x_2 - 38 \end{bmatrix}$$

Hessian (constant!)

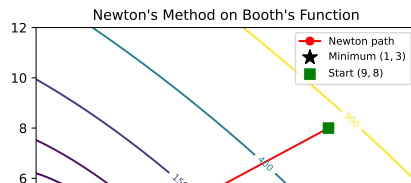
$$\mathbf{H}(\mathbf{x}) = \begin{bmatrix} 10 & 8 \\ 8 & 10 \end{bmatrix}$$

One Newton Step from $\mathbf{x}^{(1)} = [9, 8]^\top$

$$\begin{aligned} \mathbf{x}^{(2)} &= \begin{bmatrix} 9 \\ 8 \end{bmatrix} - \begin{bmatrix} 10 & 8 \\ 8 & 10 \end{bmatrix}^{-1} \begin{bmatrix} 10 \cdot 9 + 8 \cdot 8 - 34 \\ 8 \cdot 9 + 10 \cdot 8 - 38 \end{bmatrix} \\ &= \begin{bmatrix} 9 \\ 8 \end{bmatrix} - \begin{bmatrix} 10 & 8 \\ 8 & 10 \end{bmatrix}^{-1} \begin{bmatrix} 120 \\ 114 \end{bmatrix} = \begin{bmatrix} 1 \\ 3 \end{bmatrix} \end{aligned}$$

Result

Converged in **one iteration**! The Hessian is positive definite everywhere, so $\mathbf{x}^{(2)} = [1, 3]^\top$ is the global minimum.



6.2 Secant Method

Motivation

Newton's method requires the second derivative f'' . The **secant method** approximates f'' from two recent function evaluations.

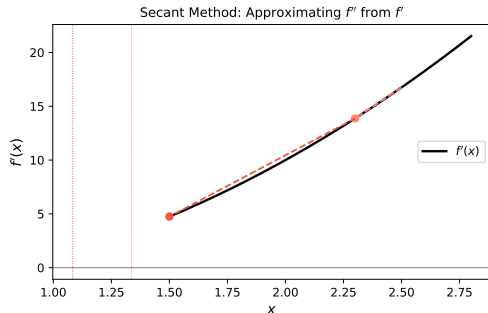
Second Derivative Approximation (Eq. 6.8)

$$f''(x^{(k)}) \approx \frac{f'(x^{(k)}) - f'(x^{(k-1)})}{x^{(k)} - x^{(k-1)}}$$

Secant Update Rule (Eq. 6.9)

Substituting into Newton's method:

$$x^{(k+1)} \leftarrow x^{(k)} - \frac{x^{(k)} - x^{(k-1)}}{f'(x^{(k)}) - f'(x^{(k-1)})} f'(x^{(k)})$$



The secant line through two evaluations of f' is used to approximate f'' and find the next iterate.

Secant Method: Properties

Algorithm 6.2: Secant Method

- ① **Input:** f' , initial $x^{(1)}$, step α , tolerance ϵ
- ② **Initialize:** $x_{\text{prev}} = x^{(1)} + \alpha f'(x^{(1)})$,
 $g_{\text{prev}} = f'(x_{\text{prev}})$
- ③ **Repeat:**
 - ① $g \leftarrow f'(x)$
 - ② $x' \leftarrow x - \frac{x - x_{\text{prev}}}{g - g_{\text{prev}}} \cdot g$
 - ③ $x_{\text{prev}}, g_{\text{prev}} \leftarrow x, g$
 - ④ $x \leftarrow x'$

Properties

- Requires only **first derivative** f'
- Needs one extra initial point to start
- Suffers the same problems as Newton's method (oscillation, negative curvature)
- May take **more iterations** than Newton's due to approximate curvature
- **Superlinear convergence** (order ≈ 1.618 , the golden ratio)

Univariate only

The secant method as stated applies to univariate objective functions. Quasi-Newton methods generalize this idea to multivariate problems.

Secant Method: Python Implementation

Listing 2: Secant method for univariate minimization in Python

```
1 import numpy as np
2
3 def secant_method(fp, x0, alpha=1e-3, eps=1e-8, max_iter=200):
4     """
5     Secant method for univariate minimization.
6     fp : first derivative f'(x)
7     x0 : initial point
8     alpha: step for initializing previous point
9     """
10    x_prev = x0 + alpha * fp(x0) # small step in gradient direction
11    g_prev = fp(x_prev)
12    x = x0
13    history = [x]
14
15    for _ in range(max_iter):
16        g = fp(x)
17        if abs(g) < eps:
18            break
19        denom = g - g_prev
20        if abs(denom) < 1e-14:
21            break # avoid division by zero
22        x_new = x - (x - x_prev) / denom * g
23        x_prev, g_prev = x, g
24        x = x_new
25        history.append(x)
```

6.3 Levenberg-Marquardt Algorithm

Motivation

Newton's method uses $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \mathbf{H}^{-1}\mathbf{g}$, which can fail when:

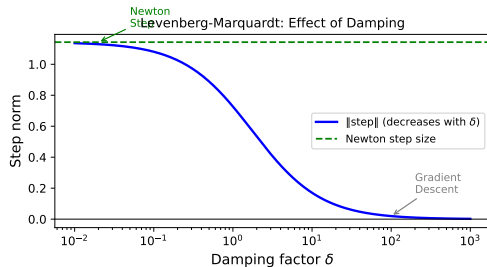
- $\mathbf{H}$ is indefinite or singular
- The quadratic approximation is poor (far from minimum)

Idea: Interpolate between gradient descent and Newton.

LM Update – Plain Damping (Eq. 6.10)

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \left(\mathbf{H}^{(k)} + \delta \mathbf{I}\right)^{-1} \mathbf{g}^{(k)}$$

- Small δ : approaches Newton's step
- Large δ : approaches gradient descent



As damping δ grows, the step shrinks from Newton size toward zero. Large δ gives gradient descent behavior.

LM Algorithm: Adaptive Damping I

Algorithm 6.3: Levenberg-Marquardt (Eq. 6.10/6.11)

Input: $f, \nabla f, \mathbf{H}, \mathbf{x}^{(1)}, \delta > 0, \gamma_{\text{acc}}, \gamma_{\text{rej}}, \epsilon$

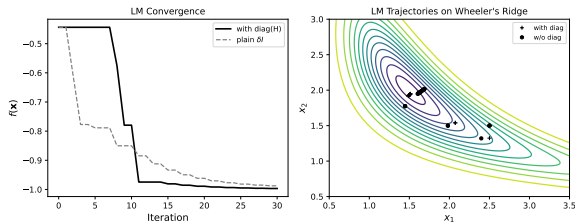
- ① Compute $\mathbf{H} = \mathbf{H}(\mathbf{x}), \mathbf{g} = \nabla f(\mathbf{x})$
- ② Form damped matrix: $\mathbf{M} = \mathbf{H} + \delta \text{diag}(\max(\text{diag}(\mathbf{H}), \epsilon))$
- ③ Compute candidate: $\mathbf{x}' = \mathbf{x} - \mathbf{M}^{-1}\mathbf{g}$
- ④ **If $f(\mathbf{x}') < f(\mathbf{x})$ (accept step):**
 - $\mathbf{x} \leftarrow \mathbf{x}'$
 - $\delta \leftarrow \delta \cdot \gamma_{\text{acc}}$ (reduce damping)
- ⑤ **Else (reject step):**
 - Keep $\mathbf{x}$ unchanged
 - $\delta \leftarrow \delta \cdot \gamma_{\text{rej}}$ (increase damping)
- ⑥ Repeat until convergence

LM Algorithm: Adaptive Damping II

Parameter Guidelines

Parameter	Meaning	Typical value
δ	initial damping factor	1.0
γ_{acc}	damping decrease factor (accept)	0.1
γ_{rej}	damping increase factor (reject)	10.0
ϵ	floor for diagonal elements	10^{-6}

LM Algorithm: Adaptive Damping III



LM convergence (left) and trajectories on Wheeler's Ridge (right).
The variant with diagonal Hessian scaling converges more efficiently.

Why Diagonal Scaling Helps

- Plain damping: adds $\delta \mathbf{I}$ uniformly
- Diagonal: adds $\delta \text{diag}(\mathbf{H})$, which is larger where curvature is larger
- Better scale-invariant behavior in flat regions
- Component-wise max with ϵ ensures the matrix is always invertible

Levenberg-Marquardt: Python Implementation

Listing 3: Levenberg-Marquardt algorithm in Python (NumPy)

```
1 import numpy as np
2 from numpy.linalg import solve
3
4 def levenberg_marquardt(f, grad_f, hess_f, x0,
5                         delta=1.0, gamma_acc=0.1, gamma_rej=10.0,
6                         eps=1e-6, max_iter=100):
7     x = np.array(x0, dtype=float)
8     history = [x.copy()]
9     for _ in range(max_iter):
10         g = grad_f(x)
11         if np.linalg.norm(g) < eps:
12             break
13         H = hess_f(x)
14         # Diagonal scaling: take component-wise max with eps
15         d = np.maximum(np.diag(H), eps)
16         M = H + delta * np.diag(d)
17         try:
18             step = solve(M, g)
19         except np.linalg.LinAlgError:
20             step = g # fallback: gradient step
21         x_new = x - step
22         if f(x_new) < f(x):
23             x = x_new
24             delta *= gamma_acc # accept: reduce damping
25     else:
```

6.4 LM for Sum of Squares

Objective Function (Eq. 6.12)

$$f(\mathbf{x}) = \sum_i f_i(\mathbf{x})^2$$

Each $f_i : \mathbb{R}^n \rightarrow \mathbb{R}$ is a *residual* function.

Gradient (Eq. 6.13)

$$\nabla f(\mathbf{x}) = 2 \sum_i f_i(\mathbf{x}) \nabla f_i(\mathbf{x})$$

Linear Approximation of Residuals (Eq. 6.14)

$$f_i(\mathbf{x}) \approx f_i(\mathbf{x}^{(k)}) + \nabla f_i(\mathbf{x}^{(k)})^\top (\mathbf{x} - \mathbf{x}^{(k)})$$

Substituting into the gradient yields an approximate gradient of the form (Eq. 6.17):

Outer Product Hessian Approximation

$$\tilde{\mathbf{g}}^{(k)} = \sum_i f_i(\mathbf{x}^{(k)}) \nabla f_i(\mathbf{x}^{(k)}) \quad (6.18)$$

$$\tilde{\mathbf{H}}^{(k)} = \sum_i \nabla f_i(\mathbf{x}^{(k)}) \nabla f_i(\mathbf{x}^{(k)})^\top \quad (6.19)$$

Note: $\tilde{\mathbf{H}}^{(k)}$ is the **Gauss-Newton** approximation (outer product of Jacobian rows), always *positive semi-definite*.

Key Advantage

No need to compute true second derivatives!
Only first derivatives ∇f_i are required.

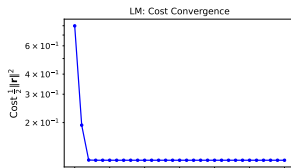
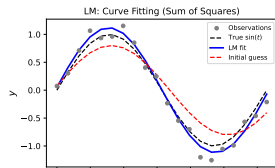
LM Sum-of-Squares: Update Formula

LM Update for Sum of Squares (Eq. 6.21)

Setting $\nabla f(\mathbf{x}) \approx 0$ and incorporating damping:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \left(\tilde{\mathbf{H}}^{(k)} + \delta \operatorname{diag}(\tilde{\mathbf{H}}^{(k)}) \right)^{-1} \tilde{\mathbf{g}}^{(k)}$$

- $\tilde{\mathbf{g}}^{(k)} = \mathbf{J}^\top \mathbf{r}$ (Jacobian times residual vector)
- $\tilde{\mathbf{H}}^{(k)} = \mathbf{J}^\top \mathbf{J}$ (Gauss-Newton Hessian, always $\succeq 0$)
- $\mathbf{J}_{ij} = \partial f_i / \partial x_j$ (Jacobian matrix)
- $\mathbf{r}_i = f_i(\mathbf{x})$ (residual vector)



Comparison: LM vs. Full Newton

	LM (sum-of-sq)	Full Newton
Hessian approx	$\mathbf{J}^\top \mathbf{J}$	exact
Always BSD?	<	No

LM for Sum of Squares: Python with SciPy

Listing 4: LM for nonlinear least squares using `scipy.optimize.least_squares`

```
1 import numpy as np
2 from scipy.optimize import least_squares
3
4 # --- Curve fitting: model  $a \sin(b \cdot t + c)$  ---
5 np.random.seed(42)
6 t_obs = np.linspace(0, 2*np.pi, 20)
7 y_obs = np.sin(t_obs) + 0.15 * np.random.randn(len(t_obs))
8
9 def model(params, t):
10     a, b, c = params
11     return a * np.sin(b * t + c)
12
13 def residuals(params):
14     return model(params, t_obs) - y_obs # vector of  $f_i(x)$ 
15
16 # SciPy uses LM (method='lm') or trust-region approaches
17 result = least_squares(residuals, x0=[0.8, 0.9, 0.1], method='lm')
18
19 print(f"Fitted params:    a={result.x[0]:.4f}, b={result.x[1]:.4f}, c={result.x[2]:.4f}")
20 print(f"True params:      a=1.0000, b=1.0000, c=0.0000")
21 print(f"Final residual:    {0.5 * result.cost:.6f}")
22 print(f"Function evals:      {result.nfev}")
23
24 # Manual approach: build Jacobian explicitly
25 def jacobian(params, t):
```

6.5 Quasi-Newton Methods: Overview

Motivation

Computing the full Hessian $\mathbf{H}^{(k)}$ at each iteration costs $O(n^2)$ evaluations and $O(n^3)$ to invert.

Quasi-Newton methods approximate the inverse Hessian $\mathbf{Q}^{(k)} \approx (\mathbf{H}^{(k)})^{-1}$ using only gradient information.

General Quasi-Newton Update

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \mathbf{Q}^{(k)} \mathbf{g}^{(k)}$$

$\mathbf{Q}^{(k)}$ is updated at each step using the **secant condition**:

$$\mathbf{Q}^{(k+1)} \boldsymbol{\delta}^{(k)} = \boldsymbol{\gamma}^{(k)}$$

where:

- $\boldsymbol{\delta}^{(k)} = \mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}$ (step vector)
- $\boldsymbol{\gamma}^{(k)} = \mathbf{g}^{(k+1)} - \mathbf{g}^{(k)}$ (gradient difference)

Three Variants

DFP Update (Algorithm 6.5)

Davidon-Fletcher-Powell Update

$$\mathbf{Q}^{(k+1)} = \mathbf{Q}^{(k)} - \frac{\mathbf{Q}^{(k)} \boldsymbol{\gamma} \boldsymbol{\gamma}^\top \mathbf{Q}^{(k)}}{\boldsymbol{\gamma}^\top \mathbf{Q}^{(k)} \boldsymbol{\gamma}} + \frac{\boldsymbol{\delta} \boldsymbol{\delta}^\top}{\boldsymbol{\delta}^\top \boldsymbol{\gamma}}$$

where $\boldsymbol{\delta} = \mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}$ and $\boldsymbol{\gamma} = \mathbf{g}^{(k+1)} - \mathbf{g}^{(k)}$.

DFP Algorithm Steps

- 1 Initialize $\mathbf{Q}^{(1)} = \mathbf{I}_n$
- 2 $\mathbf{d}^{(k)} = -\mathbf{Q}^{(k)} \mathbf{g}^{(k)}$
- 3 Line search: $\alpha^* = \arg \min_{\alpha} f(\mathbf{x}^{(k)} + \alpha \mathbf{d}^{(k)})$
- 4 $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha^* \mathbf{d}^{(k)}$
- 5 $\boldsymbol{\delta} = \mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}$, $\boldsymbol{\gamma} = \mathbf{g}^{(k+1)} - \mathbf{g}^{(k)}$
- 6 Update $\mathbf{Q}^{(k+1)}$ via DFP formula

Properties

- Maintains $\mathbf{Q} \succ 0$ (positive definite) if $\boldsymbol{\delta}^\top \boldsymbol{\gamma} > 0$ (curvature condition)
- Enforces the secant equation: $\mathbf{Q}^{(k+1)} \boldsymbol{\gamma} = \boldsymbol{\delta}$
- Rank-2 update (adds two rank-1 matrices)
- Works well with exact line search
- DFP can underperform with inexact line search

BFGS Update (Algorithm 6.6)

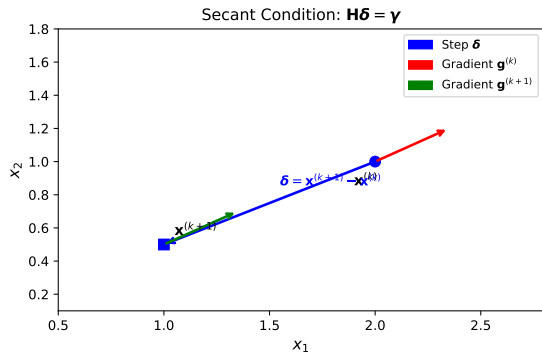
Broyden-Fletcher-Goldfarb-Shanno Update (Eq. 6.25)

$$\mathbf{Q}^{(k+1)} = (\mathbf{I} - \rho \boldsymbol{\delta} \boldsymbol{\gamma}^\top) \mathbf{Q}^{(k)} (\mathbf{I} - \rho \boldsymbol{\gamma} \boldsymbol{\delta}^\top) + \rho \boldsymbol{\delta} \boldsymbol{\delta}^\top$$

where $\rho = \frac{1}{\boldsymbol{\delta}^\top \boldsymbol{\gamma}}$.

Why BFGS > DFP?

- BFGS is the *dual* of DFP (swap roles of $\boldsymbol{\delta}$ and $\boldsymbol{\gamma}$)
- More robust with **inexact** line search
- Both are rank-2 updates
- BFGS has better numerical properties in practice
- One of the most widely used optimization algorithms



BFGS: Python Implementation

Listing 5: BFGS quasi-Newton method in Python

```
1 import numpy as np
2 from numpy.linalg import norm
3
4 def bfgs(f, grad_f, x0, eps=1e-8, max_iter=200):
5     """BFGS quasi-Newton minimization (NumPy implementation)."""
6     x = np.array(x0, dtype=float)
7     n = len(x)
8     Q = np.eye(n)          # initial inverse Hessian approx = I
9     history = [f(x)]
10
11     for _ in range(max_iter):
12         g = grad_f(x)
13         if norm(g) < eps:
14             break
15         d = -Q @ g          # descent direction
16
17         # Backtracking line search (Armijo condition)
18         alpha, c1 = 1.0, 1e-4
19         for _ in range(50):
20             if f(x + alpha*d) <= f(x) + c1*alpha*(g @ d):
21                 break
22             alpha *= 0.5
23
24         x_new = x + alpha * d
25         g_new = grad_f(x_new)
```

SR1 Update I

Symmetric Rank-1 (SR1) Update (Eq. 6.26)

$$\mathbf{Q}^{(k+1)} = \mathbf{Q}^{(k)} + \frac{(\delta - \mathbf{Q}^{(k)}\gamma)(\delta - \mathbf{Q}^{(k)}\gamma)^\top}{(\delta - \mathbf{Q}^{(k)}\gamma)^\top \gamma}$$

SR1 Properties

- **Rank-1** update (simpler than BFGS)
- Does *not* guarantee positive definiteness
- Can be useful when BFGS's rank-2 update is too restrictive
- Better at capturing non-convex structure
- Used in some trust-region frameworks

When SR1 is Undefined

The update is undefined (division by zero) when:

$$(\delta - \mathbf{Q}^{(k)}\gamma)^\top \gamma \approx 0$$

In practice: skip the update if the denominator is below a threshold.

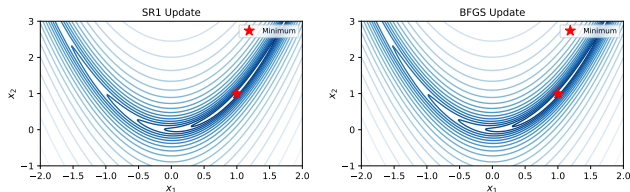
SR1 Update II

Comparison of Quasi-Newton Update Rules

Method	Rank	Always PD	Inexact LS	Memory	Convergence
SR1	1	No	Good	$O(n^2)$	superlinear
DFP	2	Yes	Poor	$O(n^2)$	superlinear
BFGS	2	Yes	Excellent	$O(n^2)$	superlinear
L-BFGS	2	Yes	Good	$O(mn)$	superlinear

PD = positive definite; LS = line search; m = history size

SR1 vs BFGS: Rosenbrock Contours



SR1 Update III

Contour plots of the Rosenbrock function. Both SR1 and BFGS can navigate curved valleys.

6.5 L-BFGS: Limited-Memory BFGS I

Motivation

BFGS stores and updates an $n \times n$ matrix, costing $O(n^2)$ memory. For $n = 10^6$, storing $\mathbf{Q}$ is infeasible. **L-BFGS** stores only the last m pairs $\{(\delta^{(i)}, \gamma^{(i)})\}$ to implicitly approximate the product $\mathbf{Q}\mathbf{g}$.

6.5 L-BFGS: Limited-Memory BFGS II

L-BFGS: Two-Loop Recursion (Alg. 6.7)

Backward pass (compute $\{\alpha^{(i)}\}$, $i = m$ down to 1):

$$\alpha^{(i)} = \frac{(\delta^{(i)})^\top \mathbf{q}^{(i)}}{(\delta^{(i)})^\top \gamma^{(i)}}, \quad \mathbf{q}^{(i-1)} = \mathbf{q}^{(i)} - \alpha^{(i)} \gamma^{(i)}$$

Starting with $\mathbf{q}^{(m)} = \nabla f(\mathbf{x})$.

Initial scaling:

$$\mathbf{z}^{(0)} = \frac{(\delta^{(m)})^\top \gamma^{(m)}}{(\gamma^{(m)})^\top \gamma^{(m)}} \mathbf{q}^{(0)}$$

Forward pass (compute $\{\mathbf{z}^{(i)}\}$, $i = 1$ to m):

$$\mathbf{z}^{(i)} = \mathbf{z}^{(i-1)} + \delta^{(i)} \left(\alpha^{(i)} - \frac{(\gamma^{(i)})^\top \mathbf{z}^{(i-1)}}{(\delta^{(i)})^\top \gamma^{(i)}} \right)$$

Backward pass (compute $\{\beta^{(i)}\}$, $i = m$ down to 1):

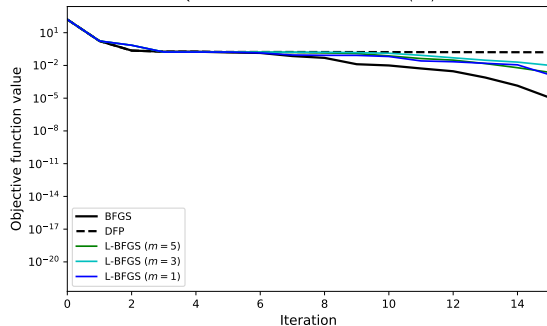
L-BFGS: Python Implementation

Listing 6: L-BFGS with two-loop recursion in Python

```
1 import numpy as np
2 from collections import deque
3
4 def lbfgs(f, grad_f, x0, m=5, eps=1e-8, max_iter=200):
5     """L-BFGS quasi-Newton minimization."""
6     x = np.array(x0, dtype=float)
7     deltas = deque(maxlen=m)      # step differences s_i
8     gammas = deque(maxlen=m)      # gradient differences y_i
9     history = [f(x)]
10
11     for _ in range(max_iter):
12         g = grad_f(x)
13         if np.linalg.norm(g) < eps:
14             break
15
16         # ---- Two-loop recursion ----
17         q = g.copy()
18         alphas = []
19         for dk, yk in zip(reversed(deltas), reversed(gammas)):
20             rho_k = 1.0 / (dk @ yk + 1e-15)
21             a = rho_k * (dk @ q)
22             q = q - a * yk
23             alphas.append((a, rho_k, dk, yk))
24         alphas.reverse()
```


Quasi-Newton Methods: Full Comparison

Quasi-Newton Methods: Rosenbrock (2D)



Objective function vs. iteration for BFGS, DFP, and L-BFGS variants ($m = 1, 3, 5$) on 2D Rosenbrock. L-BFGS approaches BFGS as m grows.

Key Observations

- BFGS and DFP often perform similarly on well-conditioned problems
- BFGS outperforms DFP with *inexact* line search
- L-BFGS($m = 1$) is weakest; L-BFGS($m \geq 5$) nearly matches BFGS
- L-BFGS is the **method of choice for large-scale** problems (e.g. deep learning, PDE-constrained optimization)

When to Use Which

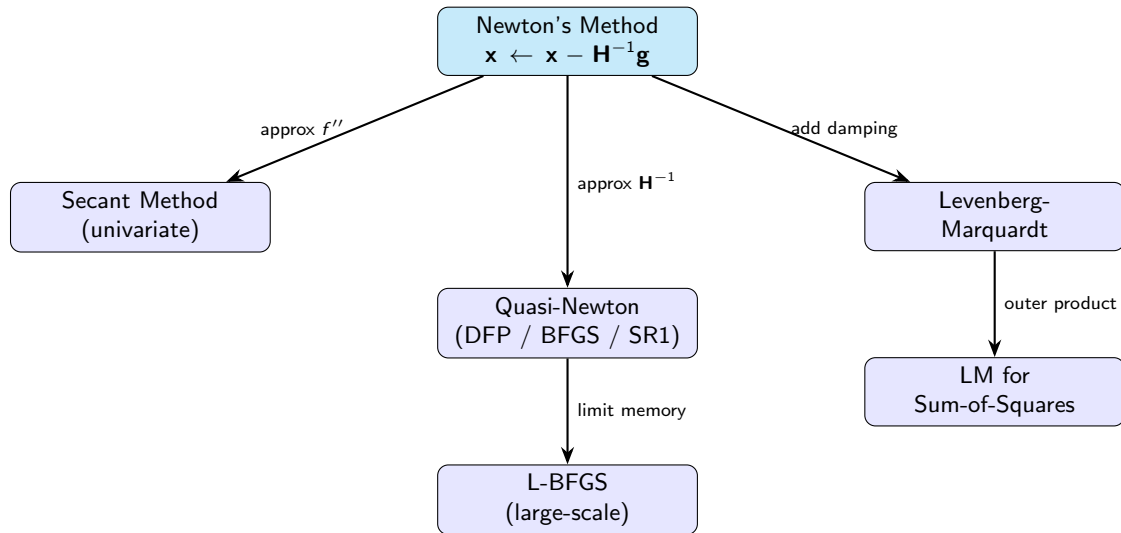
Scenario	Method
$n \leq 10^3$, cheap H	Newton
$n < 10^4$, medium	BFGS

6.6 Summary

Chapter Summary

- **Second-order information** (Hessian) enables quadratic local models that directly yield the optimal step size.
- **Newton's method** uses $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \mathbf{H}^{-1}\mathbf{g}$. Converges quadratically near a minimum; requires positive definite $\mathbf{H}$.
- **Secant method** approximates f'' from two gradient evaluations, avoiding explicit second derivatives (univariate).
- **Levenberg-Marquardt** interpolates between gradient descent (δ large) and Newton (δ small), adapting δ based on whether the step was accepted.
- **LM for sum of squares** replaces the Hessian with the outer product $\mathbf{J}^T \mathbf{J}$ (always PSD), requiring only first derivatives.
- **Quasi-Newton methods** (DFP, BFGS, L-BFGS) approximate the inverse Hessian from gradient differences, achieving superlinear convergence without computing $\mathbf{H}$.
- **L-BFGS** reduces memory from $O(n^2)$ to $O(mn)$ by storing only the last m step/gradient pairs.

Method Family Tree



6.7 Selected Exercises

Exercise 6.1

Q: What advantage does second-order information provide about convergence that first-order information lacks?

A: Second-order information can *guarantee* that one is at a local minimum (via positive definiteness of the Hessian), whereas a gradient of zero is necessary but insufficient to guarantee local optimality.

Exercise 6.9

Q: Starting from the origin, apply one step of Newton's method to $f(\mathbf{x}) = (x_1 + 1)^2 + (x_2 + 3)^2 + 4$.

A: $\nabla f = [2(x_1 + 1), 2(x_2 + 3)]^\top = [2, 6]^\top$, $\mathbf{H} = 2\mathbf{I}$.

$$\mathbf{x}^{(2)} = [0, 0]^\top - (2\mathbf{I})^{-1}[2, 6]^\top = [-1, -3]^\top = \mathbf{x}^*$$

Converges in one step since f is quadratic.

Exercise 6.7

Q: What is the advantage of a quasi-Newton method over Newton's method?

Selected Exercises (continued)

Exercise 6.2

Q: When finding roots in one dimension, when would we use Newton's method instead of the bisection method?

A: Newton's method is preferred when:

- The derivative is available and cheap to compute
- High accuracy is needed quickly (quadratic vs. linear convergence)
- The starting point is already close to the root

Bisection is preferred when derivatives are unavailable or when you need guaranteed convergence with bracketing.

Exercise 6.8

Q: Give an example where the BFGS update does not exist. What would you do?

A: The BFGS update does not exist when $\delta^\top \gamma \approx 0$ (curvature condition fails). In that case, simply **skip the update** and keep the current inverse Hessian approximation for that iteration.

Quick Reference: Second-Order Methods

Method	Update Rule	Requirements	Convergence	
Newton's Method	$\mathbf{x} \leftarrow \mathbf{x} - \mathbf{H}^{-1} \mathbf{g}$	$\nabla f, \nabla^2 f$	Quadratic	
Secant Method	$x \leftarrow x - \frac{x - x_{\text{prev}}}{g - g_{\text{prev}}} g$	f' only	Superlinear	
Levenberg-Marquardt	$\mathbf{x} \leftarrow \mathbf{x} - (\mathbf{H} + \delta D)^{-1} \mathbf{g}$	$\nabla f, \nabla^2 f$	Adaptive	
LM Sum of Squares	$\mathbf{x} \leftarrow \mathbf{x} - (\mathbf{J}^\top \mathbf{J} + \delta D)^{-1} \mathbf{J}^\top \mathbf{r}$	$\mathbf{J}$ only	Adaptive	$D = \text{diag}(\mathbf{H})$
DFP	$\mathbf{Q} \leftarrow \mathbf{Q} - \frac{\mathbf{Q} \gamma \gamma^\top \mathbf{Q}}{\gamma^\top \mathbf{Q} \gamma} + \frac{\delta \delta^\top}{\delta^\top \gamma}$	∇f only	Superlinear	
BFGS	$\mathbf{Q} \leftarrow (\mathbf{I} - \rho \delta \gamma^\top) \mathbf{Q} (\mathbf{I} - \rho \gamma \delta^\top) + \rho \delta \delta^\top$	∇f only	Superlinear	
L-BFGS	Two-loop recursion with m history pairs	∇f only	Superlinear	

or $\mathbf{I}$; $\rho = 1/(\delta^\top \gamma)$